BITS Pilani, M.Tech (AIML) | AIMLCZG523 MLOps | Assignment 01

End-to-End MLOps Pipeline for Heart Disease Prediction

From data and experiment tracking to CI/CD, Docker, Kubernetes and monitoring

Item	Details
Student	Vikrant Vilas Vankudre
BITS ID	2025ae05336
Course	MLOps (AIMLCZG523), Semester 3
Dataset	UCI Heart Disease (Cleveland), 303 patients, 13 features
Code repository	https://github.com/vankudrv/heart-disease-mlops
Video (full quality)	https://drive.google.com/file/d/1l1NyGt8uGgFhaOEbg-KFtx0vx-AS1yjG/view?usp=drive_link
Video (compressed, in repo)	https://github.com/vankudrv/heart-disease-mlops/raw/main/video/MLOPS_Assignment1_2025AE05336.mp4
Date	20 October 2026

Summary. I built a complete, reproducible MLOps pipeline around a heart-disease classifier. Three models were tuned with cross-validation and tracked in MLflow; the selected model (logistic regression, hold-out ROC-AUC 0.967, recall 0.929) is packaged, served by a FastAPI service in a non-root Docker image, tested and built by GitHub Actions, deployed to a two-replica Minikube Deployment behind a LoadBalancer Service, and observed with Prometheus, Grafana and structured JSON logs. Every step in this report is backed by a screenshot taken from my own run, and all code is in the public repository above.

1. Problem statement and approach

The task is to predict whether a patient has heart disease from routine clinical measurements, and to wrap the model in the engineering practices expected of a production ML system: versioned code, tracked experiments, automated tests, containerisation, orchestration and monitoring. The target is binary: the original UCI label `num` (0 to 4) is converted to `target = 1` when any heart disease is present.

Design decisions that shaped the project:

- **Recall matters more than accuracy.** In a screening setting a missed patient is costlier than a false alarm, so ties between similar models are broken by recall.
- **No data leakage.** All imputation, scaling and encoding live inside one scikit-learn Pipeline that is fitted only on training folds.
- **One artefact, same everywhere.** The same saved pipeline file is used by the training script, the tests, the Docker image and the Kubernetes pods, and its dependencies are pinned to the training environment (scikit-learn 1.6.1).
- **Everything scripted.** Each stage can be re-run from the command line, and the CI workflow re-runs lint, tests and training on every push and pull request.

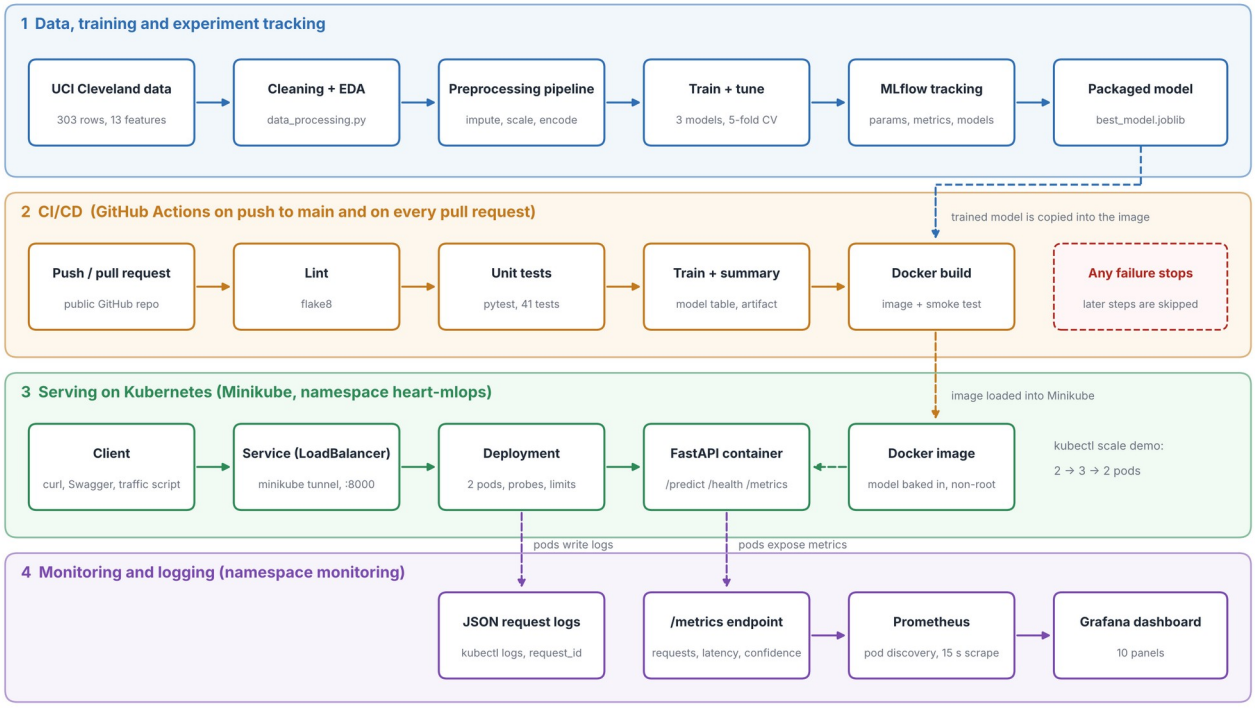


Figure 1. *End-to-end architecture: training and tracking, CI/CD, Kubernetes serving, and monitoring.*

2. Environment setup and repository

Development was done on Windows 11 with Python 3.11.9 in VS Code. The project lives in a single public GitHub repository (https://github.com/vankudrv/heart-disease-mlops) and is laid out as follows.

Path	Purpose
src/data_processing.py, scripts/download_data.py	Download the UCI file, clean it, write data/processed/heart_clean.csv
src/preprocessing.py	Feature engineering and the imputation / scaling / encoding pipeline
src/train.py, src/predict.py	Tuning, evaluation, MLflow logging, model saving; inference helper
src/api/	FastAPI app, request/response schemas, JSON logging, Prometheus metrics
tests/	pytest suite: data, preprocessing, model and API tests (41 test cases)

Path	Purpose
Dockerfile, requirements*.txt	Container image and pinned dependencies
.github/workflows/ci.yml	CI pipeline: lint, test, train, Docker build and smoke test
k8s/, monitoring/	Kubernetes manifests; Prometheus and Grafana configuration
models/, reports/, notebooks/, screenshots/	Saved model + metadata, figures, EDA notebook, evidence

Table 1. *Repository layout.*

Setup from a clean machine takes four commands in PowerShell:

```
python -m venv .venv ; .venv\Scripts\activate ; pip install -r requirements.txt
python scripts/download_data.py ; python src/data_processing.py
python -m src.train        # trains 3 models, logs to MLflow, saves
models/best_model.joblib
```

The README documents the remaining commands (API, Docker, Kubernetes, monitoring). Windows-specific problems met on the way (PATH after installing Python, PowerShell execution policy, VS Code Restricted Mode, WSL for Docker Desktop) were resolved and are noted in Section 13.

3. Data acquisition, cleaning and EDA

The Cleveland subset of the UCI Heart Disease dataset (`processed.cleveland.data`) is downloaded by script. It has 303 patients and 13 predictors: five numeric (age, resting blood pressure, cholesterol, maximum heart rate, ST depression) and eight categorical or ordinal (sex, chest-pain type, fasting blood sugar, resting ECG, exercise angina, slope, number of coloured vessels, thalassaemia). Missing values are coded as "?"; they occur in **ca (4 rows)** and **thal (2 rows)**. No duplicate rows were found. The binarised target is reasonably balanced: **164 patients without disease (54%) and 139 with disease (46%)**, so accuracy is not misleading and no resampling was needed.

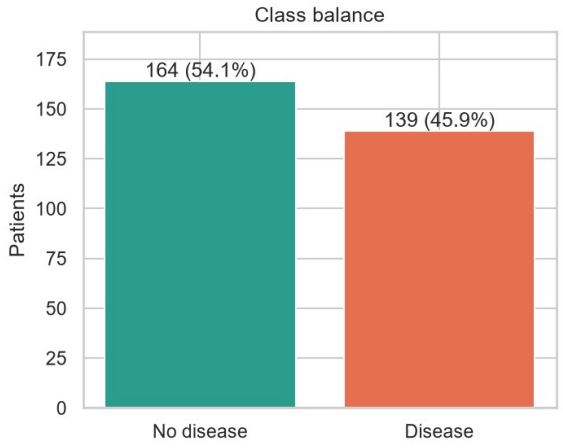


Figure 2. *Class balance of the binary target.*

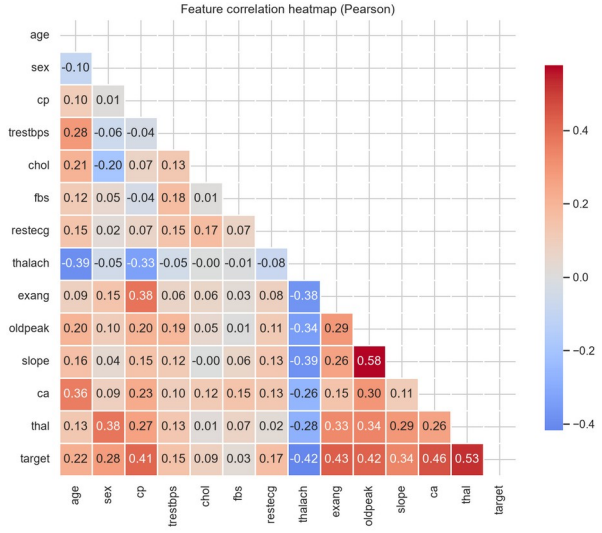


Figure 3. *Correlation heatmap of the features.*

Findings that guided the modelling:

- **Strongest signals:** thalassaemia type (r = 0.53 with the target), number of coloured vessels (0.46), exercise-induced angina (0.43), ST depression (0.42) and chest-pain type (0.41). Maximum heart rate is the strongest *negative* signal (-0.42): patients with disease reach lower peak heart rates.
- **Weak signals:** cholesterol (0.09) and fasting blood sugar (0.03) carry little linear information on their own.
- **Outliers and scale:** cholesterol reaches 564 mg/dl and ST depression 6.2; features sit on very different scales, which justifies standardisation for the linear model.

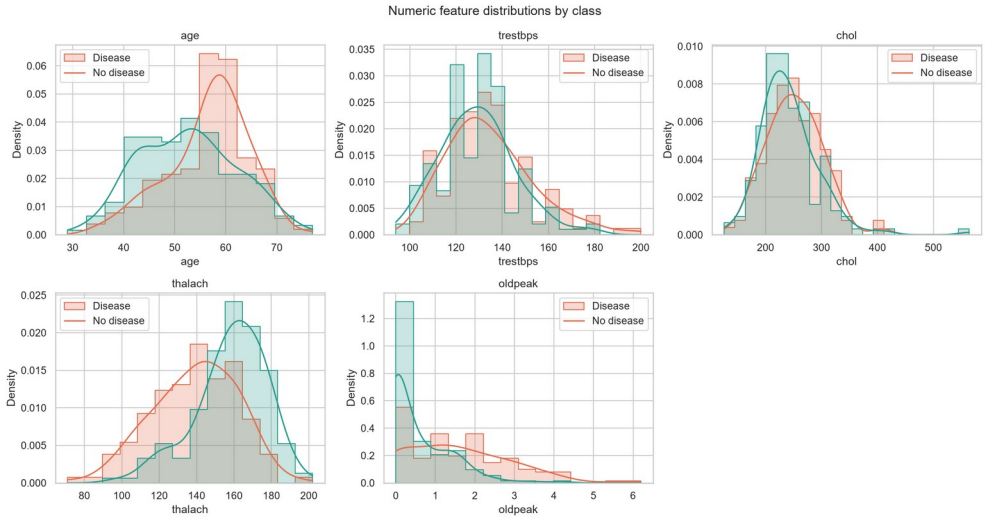


Figure 4. *Distributions of the numeric features.*

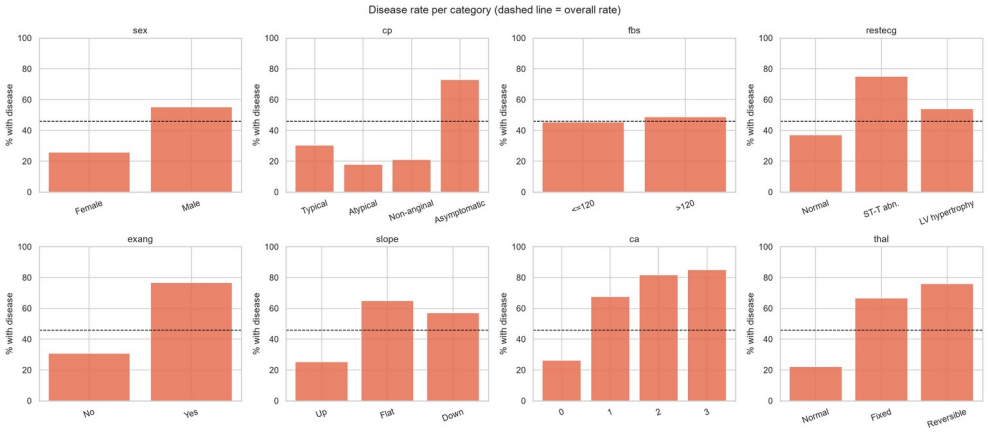


Figure 5. *Disease rate by category of each categorical feature.*

4. Feature engineering and model development

4.1 Preprocessing pipeline

A scikit-learn `ColumnTransformer` is placed in front of every classifier: numeric columns are median-imputed and standardised; categorical columns are imputed with the most frequent value and one-hot encoded. One engineered feature, `hr_pct_max = thalach / (220 - age)`, expresses the peak heart rate as a fraction of the age-predicted maximum. An ablation run (`python -m src.train --no-fe --no-save`) showed no measurable gain from it, so it is kept as a documented, harmless domain feature rather than claimed as an improvement.

4.2 Models, tuning and evaluation protocol

The data is split 80/20 with stratification (242 training and 61 test rows, seed 42). Three models were tuned on the training split with 5-fold stratified `GridSearchCV` scored by ROC-AUC, and reported with accuracy, precision, recall, F1 and ROC-AUC. The hold-out set was used once, for the final comparison.

Model	Search space	Best parameters
Logistic regression	C in {0.01, 0.1, 1, 10}; class_weight in {None, balanced} (8 combinations)	C = 1, class_weight = None
Random forest	trees {100, 300}; depth {3, 5, None}; min leaf {1, 3}	100 trees, depth 5, min leaf 3
Gradient boosting	trees {100, 200}; learning rate {0.05, 0.1}; depth {2, 3}	100 trees, rate 0.05, depth 2

Table 2. *Hyper-parameter search spaces and selected values.*

4.3 Results and model selection

Model	CV acc.	CV recall	CV ROC-AUC	Test acc.	Test recall	Test F1	Test ROC-AUC
Random forest	0.810	0.746	0.903	0.852	0.893	0.847	0.955
Logistic regression	**0.847**	**0.783**	**0.902**	**0.885**	**0.929**	**0.881**	**0.967**
Gradient boosting	0.822	0.783	0.878	0.902	0.929	0.897	0.959

Table 3. *Cross-validation (5-fold, training split) and hold-out test metrics. Selected model highlighted.*

Logistic regression and random forest are tied on cross-validated ROC-AUC (0.902 versus 0.903, well inside one standard deviation of about 0.014 to 0.024). I therefore pre-defined the selection rule **"CV ROC-AUC within 0.01 of the best counts as tied; among tied models pick the highest CV recall"**, which selects **logistic regression** (CV recall 0.783 versus 0.746). It also has the best hold-out ROC-AUC (0.967) and ties for the best recall (0.929), and as a linear model it is simple to explain and fast to serve.

Caveat. The test set has only 61 patients, so a single misclassified patient moves recall by about 3.5 points; the three models are statistically close, and the choice should be read as "a well-regularised linear model is sufficient here" rather than "logistic regression is decisively best".

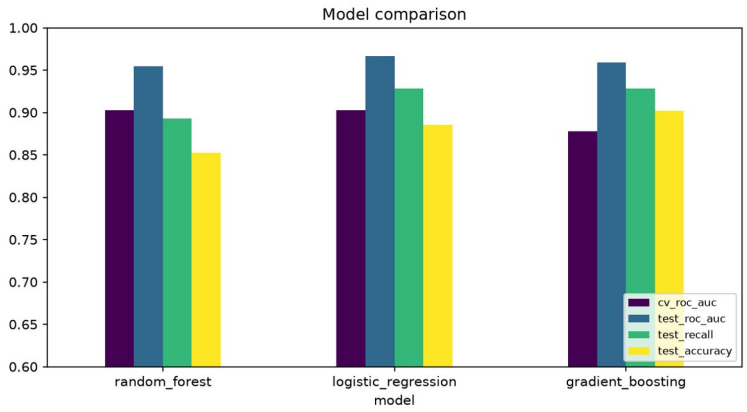


Figure 6. *Model comparison across CV ROC-AUC and hold-out ROC-AUC, recall and accuracy.*

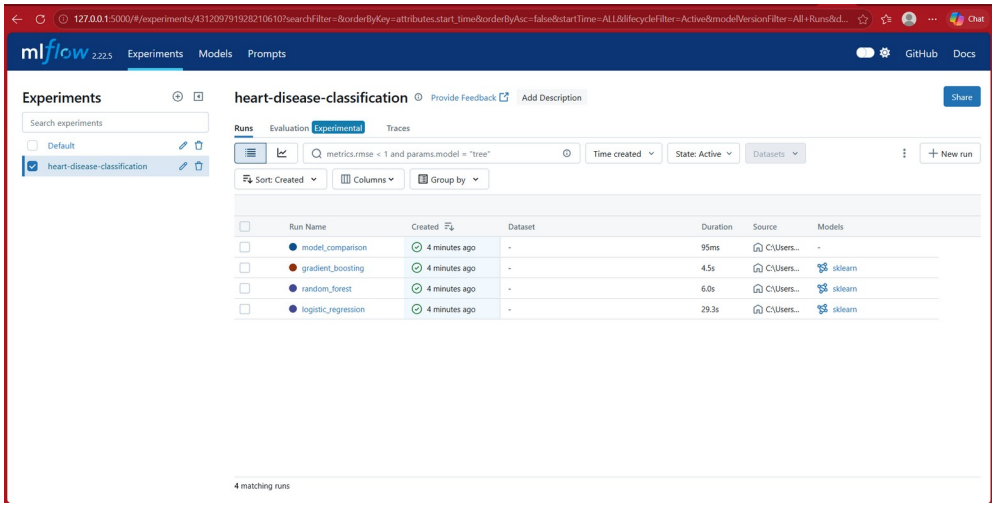


Figure 7. *Console output of python -m src.train: per-model CV scores, tie rule, selected model.*

5. Experiment tracking with MLflow

Every training run is logged to a local MLflow tracking store (`./mlruns`, MLflow 2.22) under the experiment `heart-disease-classification`. Each of the three model runs records its parameters (best hyper-parameters, number of CV folds, grid size, split sizes, seed, whether the engineered feature is used), cross-validation and hold-out metrics, a confusion matrix and ROC curve as artefacts, a text classification report, and the fitted pipeline as an MLflow model with input signature and example. A fourth

run, `model_comparison`, stores the comparison table and the selection decision. This makes any result reproducible from its run ID.

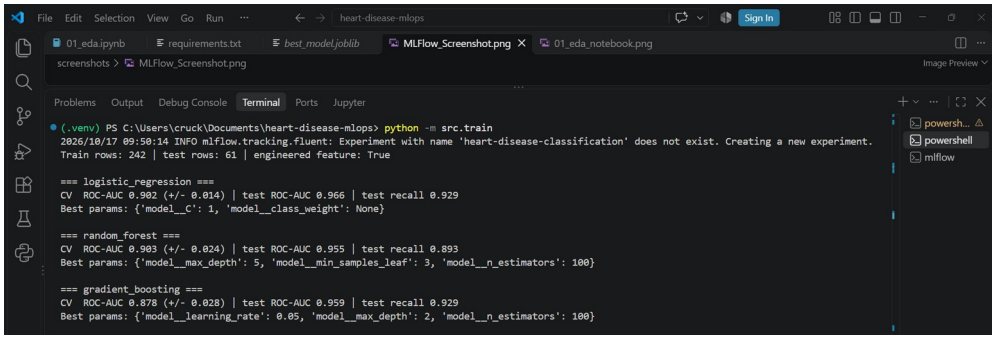


Figure 8. *MLflow experiment with the three model runs and the model_comparison run.*

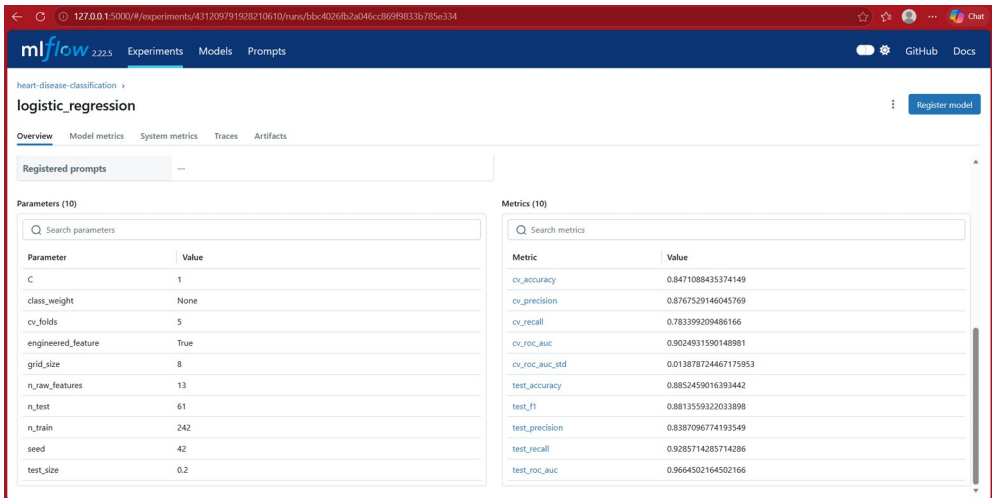


Figure 9. *Run detail for the selected model: 10 parameters and 10 metrics.*

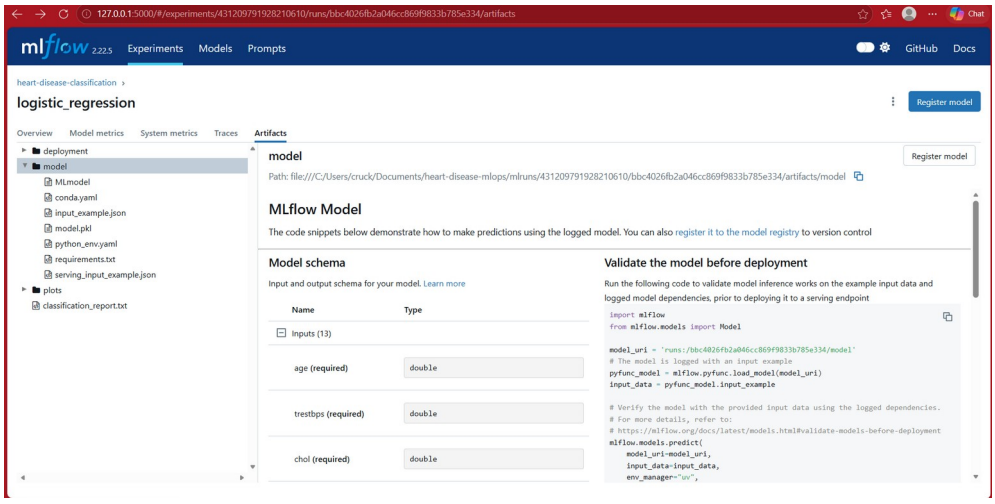


Figure 10. *Logged artefacts: MLflow model (with schema and pinned requirements) and plots.*

6. Model packaging and reproducibility

The winning pipeline is saved with joblib as `models/best_model.joblib` together with `models/model_metadata.json` (model name, selection rule, best parameters, metrics, the 13 input features in order, scikit-learn version, MLflow run ID and training timestamp). Because the preprocessing is inside the saved object, the API only needs to pass the 13 raw fields. Three requirement files separate concerns: `requirements.txt` (training and tracking), `requirements-dev.txt` (adds pytest, httpx, flake8) and `requirements-api.txt` (slim, **pinned** serving dependencies generated from the training

environment by `scripts/pin_api_requirements.py`). Pinning matters: a pickled scikit-learn model can fail or silently change behaviour under a different version.

7. Model serving API

The model is served by FastAPI with these endpoints: `POST /predict` (returns prediction, label, confidence, probability of disease, model name and a request ID), `GET /health` (503 until the model is loaded, used by Docker and Kubernetes probes), `GET /model-info`, `GET /metrics` (Prometheus) and the automatic Swagger UI at `/docs`. Input is validated with Pydantic: ranges are enforced (for example age 1 to 120, chest-pain type 1 to 4), required fields cannot be omitted, and `ca` and `thal` are optional and imputed as in training. Invalid input returns HTTP 422 with a precise message instead of a model error.

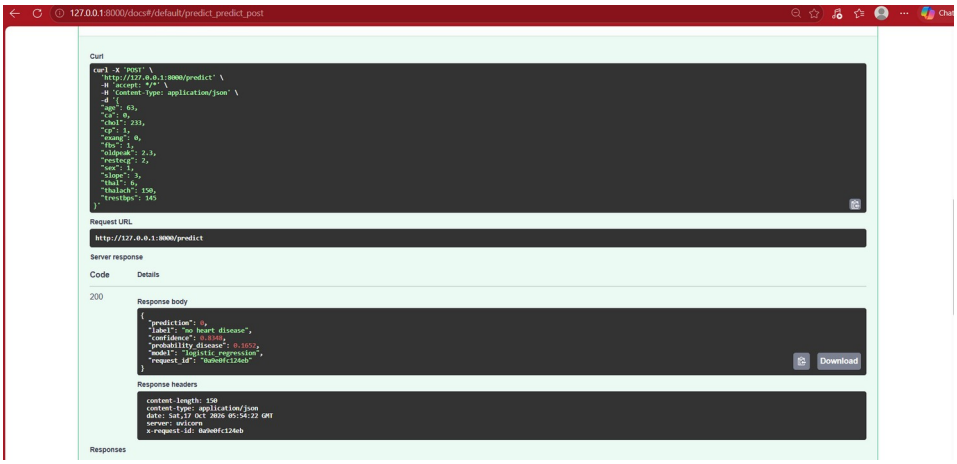


Figure 11. *POST /predict with a valid patient: HTTP 200, prediction, label and confidence.*

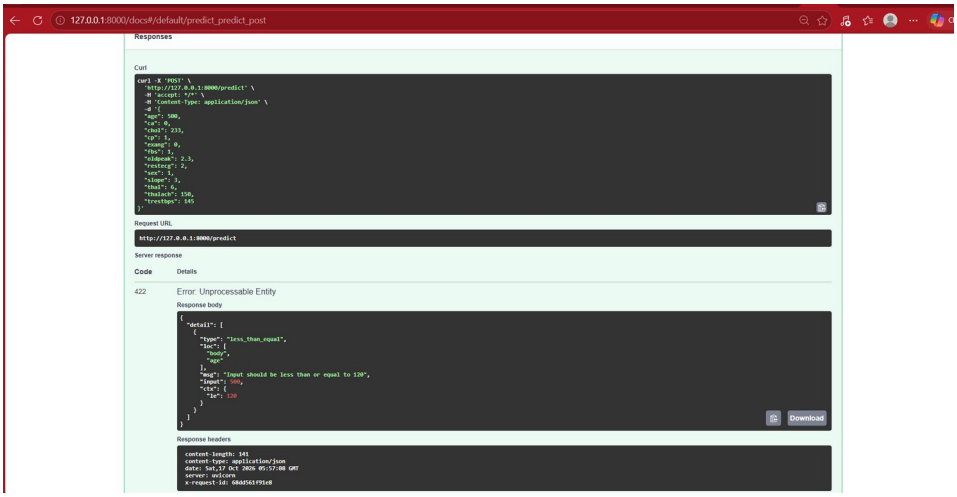


Figure 12. *Input validation: an out-of-range value is rejected with HTTP 422.*

8. Automated testing and CI/CD

8.1 Unit tests and lint

The pytest suite contains **41 test cases** in four files. **Data tests** check the raw file shape, that "?" becomes a missing value, that the target is binary and the original label is dropped, that duplicates are removed, that only `ca` and `thal` have missing values and that the classes are not extremely imbalanced. **Preprocessing tests** check the fixed column order, the engineered feature (correct value, safe for extreme ages, input not modified), that no missing values remain, that numeric columns are standardised and that an unseen category does not crash the pipeline. **Model tests** check output fields and ranges, that label and confidence agree with the probability, batch prediction, a clear error for a missing feature, that and `thal` may be missing, that a high-risk patient scores higher than a low-risk one, that the model clearly beats chance, and that a saved model reloads with identical predictions. **API tests** check health,

prediction, optional fields, missing and invalid input (parametrised), model-info, metrics, and HTTP 503 when the model file is missing. Code style is checked by flake8 with a repository-level configuration.

8.2 GitHub Actions workflow

The workflow `.github/workflows/ci.yml` runs on every push to `main` and on every pull request, in two jobs. **Lint, test and train**: checkout, Python 3.11, install dependencies, flake8, pytest with a JUnit report, training with MLflow, a model-comparison table written to the job summary, and upload of the models, reports and mlruns folder as the `training-outputs` artefact. **Build and smoke-test the Docker image** (runs only if the first job passes): builds the image, starts the container, waits for `/health` and posts a sample patient to `/predict`.

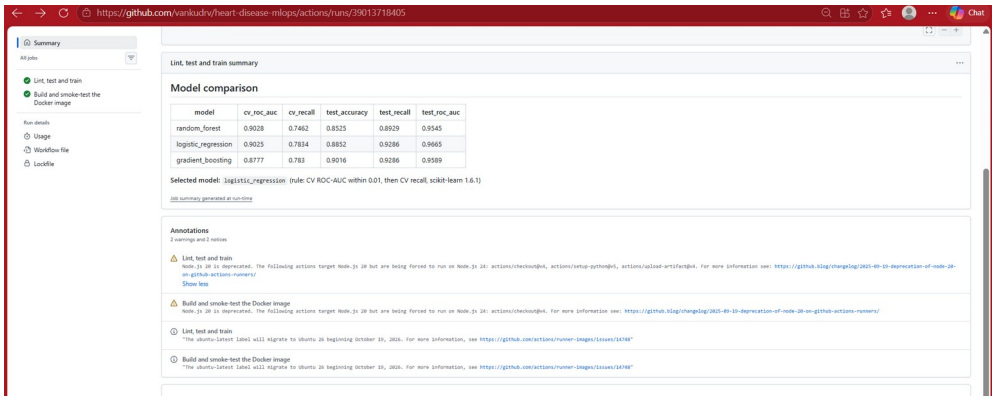


Figure 13. *Successful run: job graph, model-comparison table in the job summary, and uploaded artefact.*

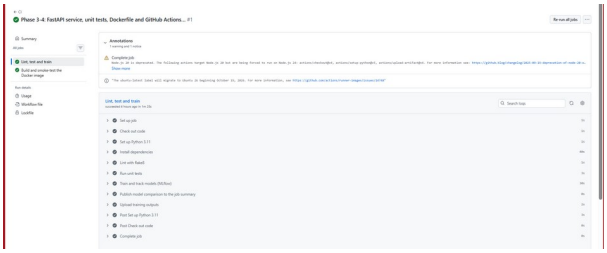


Figure 14. *Steps of the lint, test and train job.*

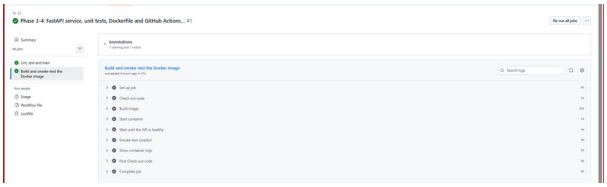


Figure 15. *Steps of the Docker build and smoke-test job.*

8.3 Demonstrating that the pipeline blocks bad code

To prove that the gate works, I opened a pull request from a throw-away branch (`demo/failing-ci`) that added one deliberately failing test. GitHub Actions failed the unit-test step with the assertion message, skipped model training and the Docker job, and marked the pull request as having failing checks. The pull request was closed without merging and the branch deleted locally, so `main` stays green.

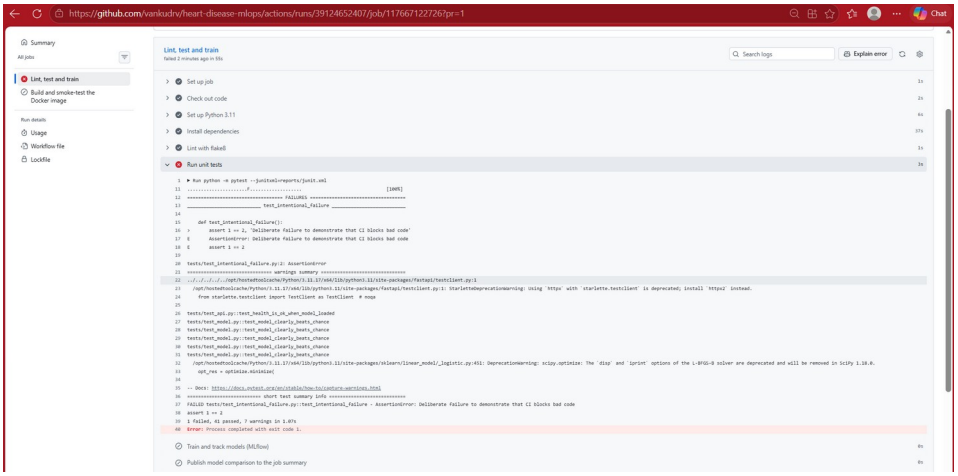


Figure 16. *Failing run: 41 tests passed and 1 failed; later steps are skipped.*

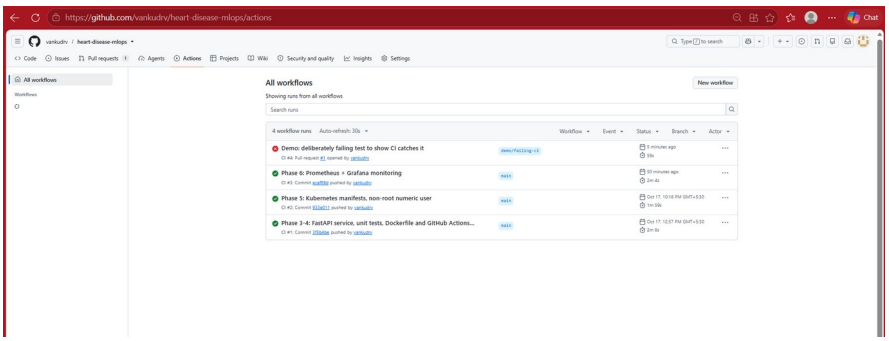


Figure 17. *Red run for the pull request above the green runs on main.*

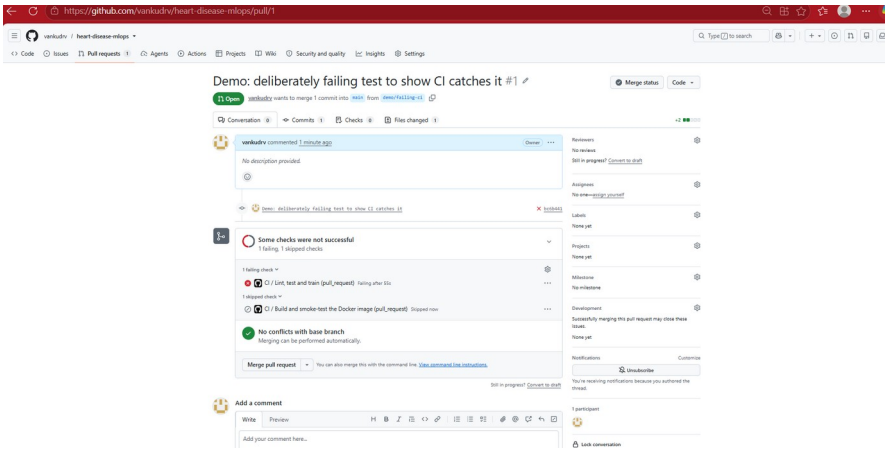


Figure 18. *Pull request showing "Some checks were not successful".*

9. Containerisation

The `Dockerfile` builds on `python:3.11-slim`, installs only the pinned serving dependencies, and copies just the API code and the trained model (data, notebooks, tests and manifests are excluded through `.dockerignore`). The container runs as the non-root numeric user **UID 10001** (a numeric UID lets Kubernetes verify `runAsNonRoot`), exposes port 8000, and defines a `HEALTHCHECK` on `/health`. The resulting image is 164 MB of content (719 MB on disk). It was built and run locally and tested with the same request as the local API before moving to Kubernetes.

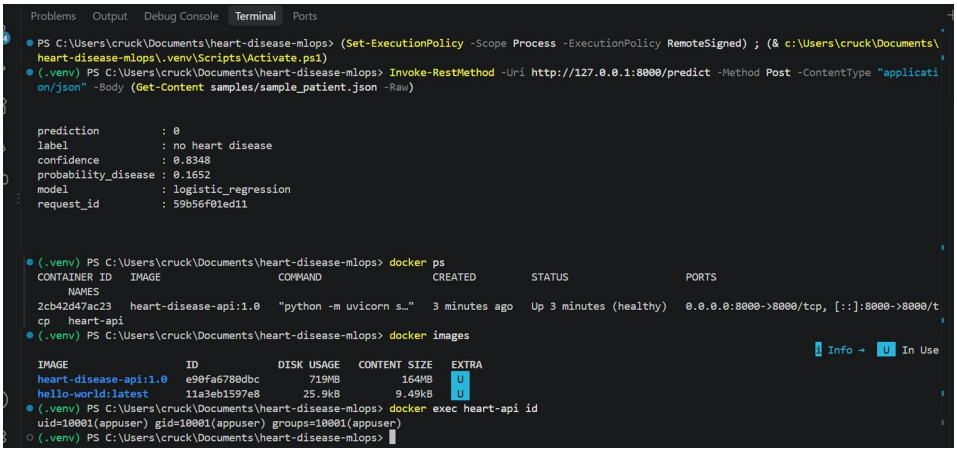


Figure 19. *Container healthy, /predict served from the container, image listed, and process running as appuser (UID 10001).*

10. Kubernetes deployment (Minikube)

The cluster is a single-node Minikube (Docker driver). The image is loaded into the cluster with `minikube image load heart-disease-api:1.0` and everything is applied with one command, `kubectl apply -k k8s/` (Kustomize). The manifests define:

Object	Key settings
Namespace heart-mlops	Isolates the application from monitoring and system components
Deployment heart-api	2 replicas; readiness and liveness probes on /health; requests 100m CPU / 256 Mi, limits 500m / 512 Mi; runs as UID 10001 without privilege escalation; imagePullPolicy IfNotPresent (local image)
Service heart-api	Type LoadBalancer, port 8000 to the container port named http; external IP provided by minikube tunnel (127.0.0.1)
HorizontalPodAutoscaler (optional)	2 to 4 replicas at 70% CPU; kept as a separate file, not applied by default

Table 4. *Kubernetes objects defined in k8s/.*

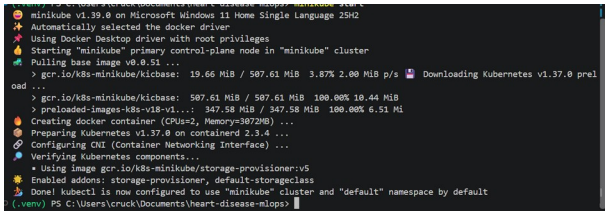


Figure 20. *Minikube cluster started.*



Figure 21. *Image loaded, manifests applied, pods becoming Ready.*

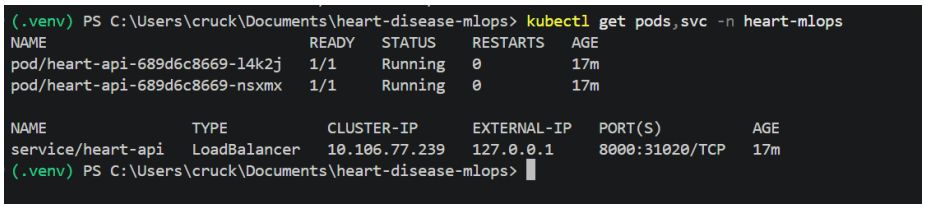


Figure 22. *Two pods Running and the LoadBalancer service with external IP 127.0.0.1.*



Figure 23. *Prediction served by the pods through the LoadBalancer service.*

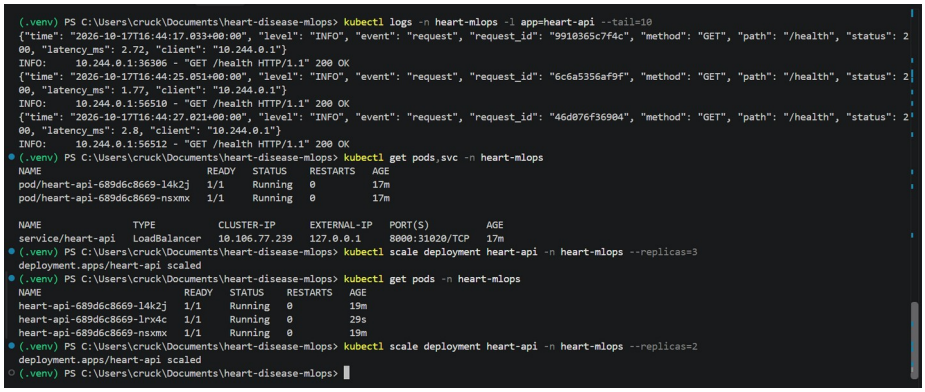


Figure 24. *Scaling demonstration: replicas 2 to 3 (new pod Running within seconds) and back to 2.*

11. Monitoring and logging

Application metrics. The API exports Prometheus metrics: `api_requests_total` (by method, endpoint and status), `api_request_latency_seconds` (histogram per endpoint), `predictions_total` (by predicted class), `prediction_confidence` (histogram) and `model_loaded` (gauge).

Prometheus and Grafana. Both run in a separate `monitoring` namespace and are deployed with `kubectl apply -k monitoring/`. Prometheus discovers the API pods through the Kubernetes API (a namespaced Role allows it to list pods only in heart-mlops) and scrapes each pod every 15 seconds, so replicas appear and disappear automatically when the Deployment is scaled. Grafana is provisioned from ConfigMaps with the Prometheus data source and a 10-panel dashboard: pods up, model loaded, predictions, average confidence, error responses, request rate per endpoint, p50 and p95 latency of /predict, predictions by class, status codes and requests per pod.

Logging. Each request produces one JSON log line with timestamp, request ID, method, path, status, latency and client address, readable with `kubectl logs`. Patient inputs are not logged unless explicitly enabled, to avoid writing health data into logs.

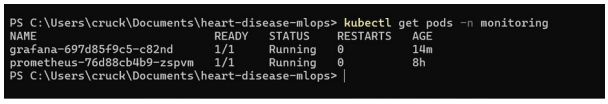


Figure 25. *Monitoring pods Running.*



Figure 26. *Traffic generator: 300 requests (286 OK, 14 invalid).*

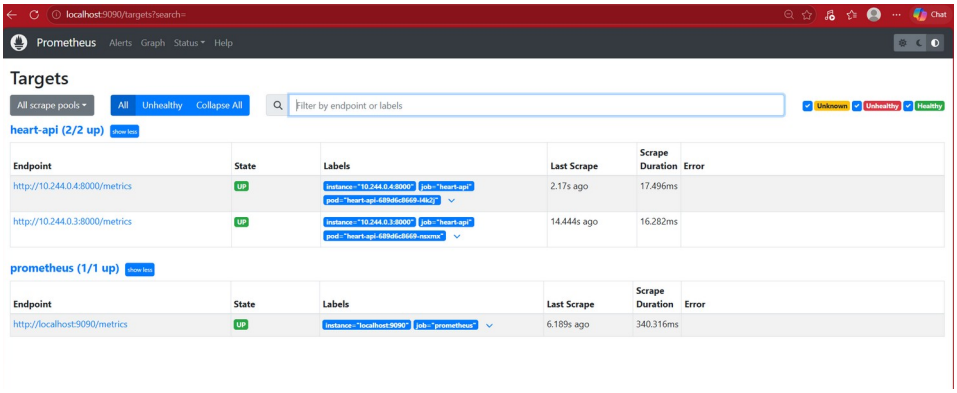


Figure 27. *Prometheus targets: both API pods discovered and UP.*

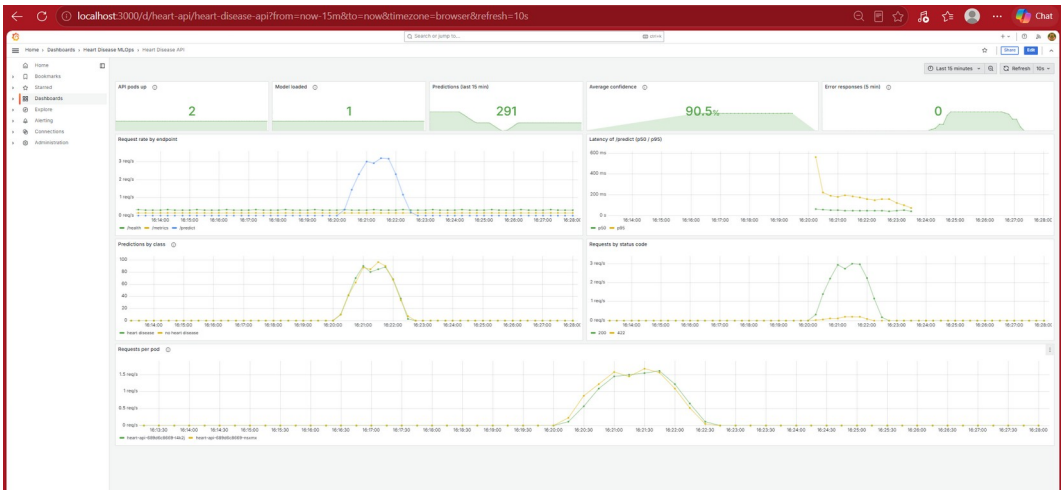


Figure 28. *Grafana dashboard after a traffic run: both pods share the load, p95 latency below about 500 ms, two predicted classes, 200 and 422 responses.*



Figure 29. *Request logs from the pods showing POST /predict calls with latency.*

12. Verification on the BITS lab virtual machine

To show that the project is reproducible on a machine other than my own laptop, I cloned the public repository on the BITS Prayogshala lab VM and ran the pipeline there end to end, following the README with bash instead of PowerShell. Most screenshots in this section show the lab session title bar with my user name (s2025ae05336) and the lab clock (18 October 2026). The original screenshots (files `lab_02_*` to `lab_12_*`) are in the `screenshots/` folder of the repository; some are cropped here to save space.

Item	Lab VM
Operating system	Rocky Linux 9.5 (browser-based remote desktop)
Resources	4 vCPU, 15 GiB RAM; Minikube started with 2 CPUs and 4 GiB
Python	3.12.9 (the laptop used 3.11.9); virtual environment inside ~/workspace
Docker / Kubernetes	Docker 28.0.1; Minikube 1.35.0 running Kubernetes v1.32.0
kubectl	Not installed; used through a one-line wrapper around minikube kubectl
Storage	Only ~/workspace is persistent; everything else is lost when the VM stops

Table 5. *Lab VM environment.*

What was checked and what I found.

- **Tests and lint.** The repository installed from the same requirement files on Python 3.12 without changes; the pytest suite passed (100%) and flake8 reported no issues.
- **Training reproduced.** Training on the lab VM selected the same model by the same rule: logistic regression with CV ROC-AUC 0.902, test accuracy 0.885 and test recall 0.929. The other metrics matched except hold-out ROC-AUC, 0.966 here against 0.967 on the laptop, a difference of 0.001 that I attribute to different library versions. The MLflow UI, started on port 5000, lists the three model runs and the comparison run.
- **Model file kept consistent.** The model trained on the lab VM was built with the lab's package versions, so I restored the committed model (`git checkout -- models/`) before building the image. This keeps the model and the pinned serving dependencies consistent.
- **Docker.** The image built from the Dockerfile in 45 seconds (505 MB as listed by docker images on the lab VM), the container became healthy, answered /health and /predict, and ran as UID 10001.
- **Kubernetes.** Minikube started with the Docker driver, the image was loaded, `kubectl apply -k k8s/` created the namespace, service and two replicas, and a prediction was served through the service. Without `minikube tunnel` the LoadBalancer external IP stays pending, so I reached the service with `kubectl port-forward` on port 8000. Scaling to three replicas and back worked as on the laptop.
- **Monitoring.** `kubectl apply -k monitoring/` started Prometheus and Grafana. After a 300-request traffic run (286 valid, 14 invalid), Prometheus showed both API pods UP, and Grafana showed 2 pods up, 259 predictions and 91.8% average confidence in the last 15 minutes. The error-responses panel counts only the last 5 minutes, so it had returned to 0 when the screenshot was taken; the status-code panel still shows the 422 responses caused by the invalid requests.

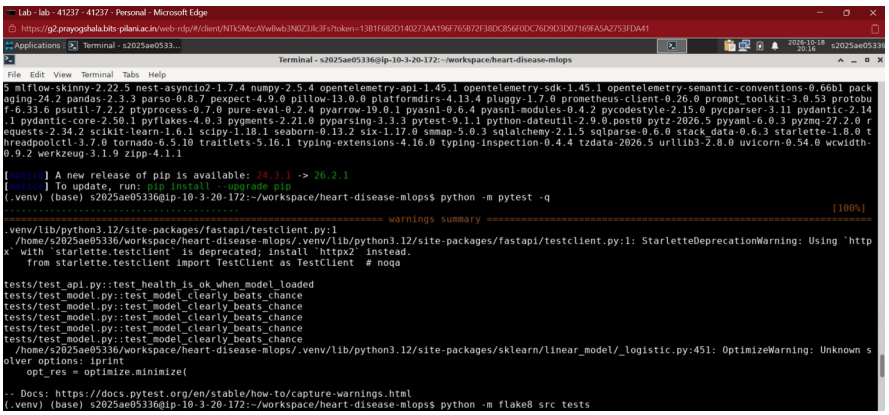


Figure 30. *Lab VM: pytest passes (100%) and flake8 is run on src and tests.*

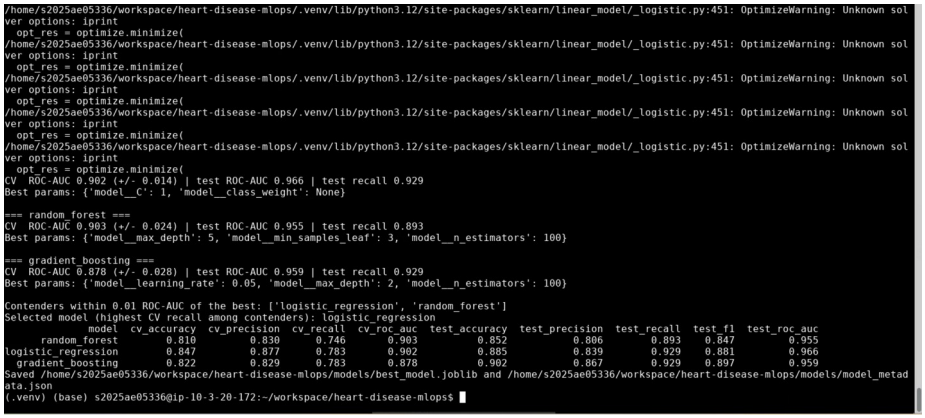


Figure 31. *Lab VM: training selects logistic regression by the tie rule, with the same metrics as on the laptop.*

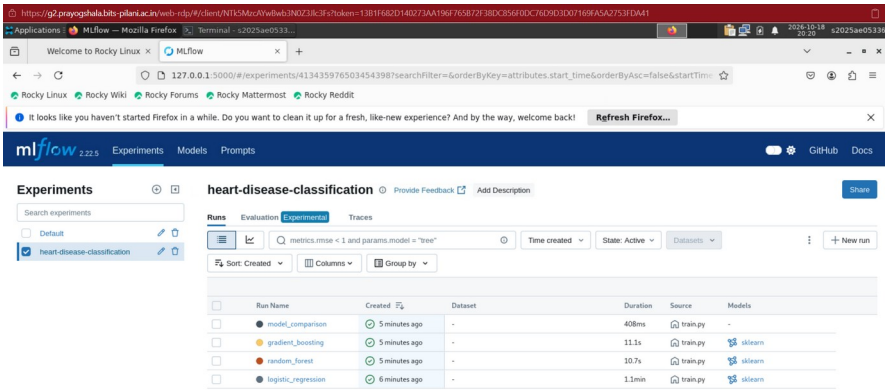


Figure 32. *Lab VM: MLflow UI with the three model runs and the model_comparison run.*

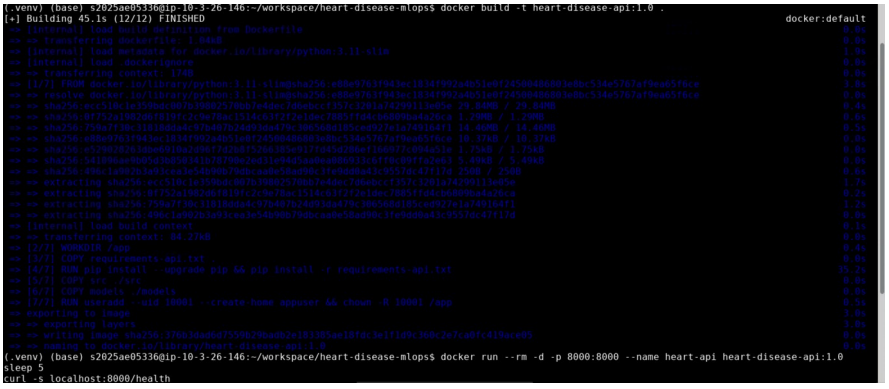


Figure 33. *Lab VM: Docker image build (45 s) and container start.*

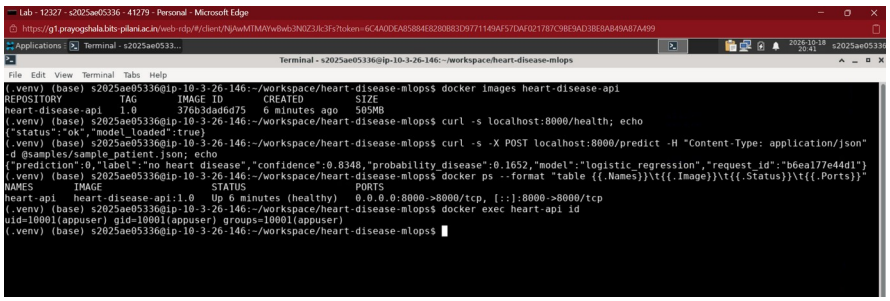


Figure 34. *Lab VM: image listed, /health and /predict answered, container healthy and running as UID 10001.*

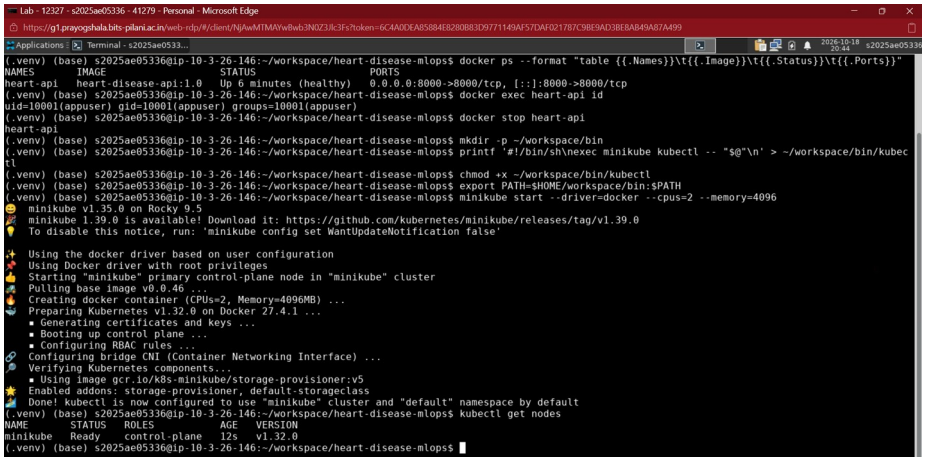


Figure 35. *Lab VM: container health check, non-root user, then Minikube started and the node Ready.*

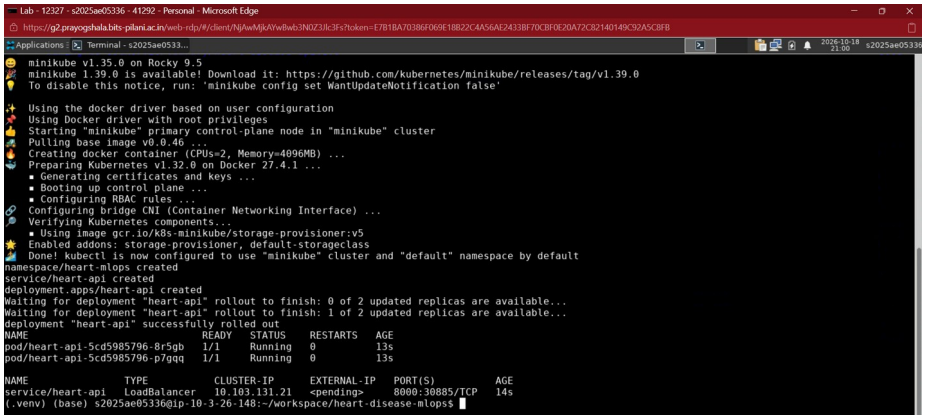


Figure 36. *Lab VM: manifests applied; both pods 1/1 Running and the LoadBalancer service created.*

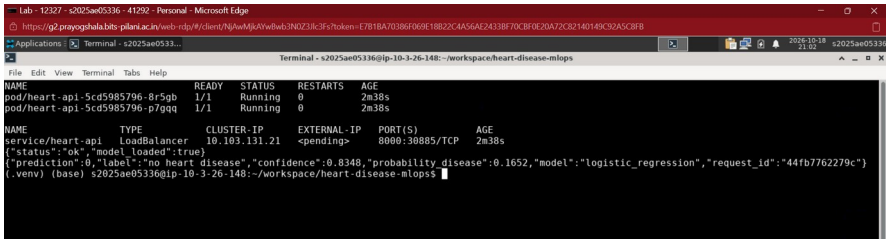


Figure 37. *Lab VM: pods, service and a prediction through the Kubernetes service.*

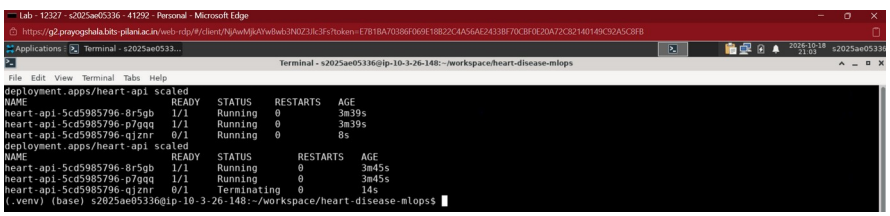


Figure 38. *Lab VM: scaling from 2 to 3 replicas and back to 2.*

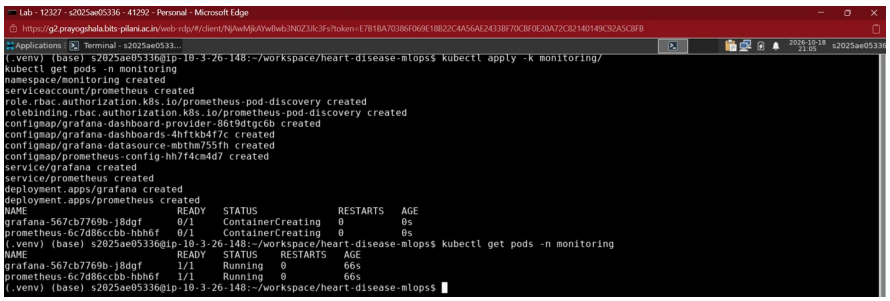


Figure 39. *Lab VM: monitoring manifests applied; Prometheus and Grafana pods Running.*



Figure 40. *Lab VM: traffic generator, 300 requests (286 OK, 14 invalid).*

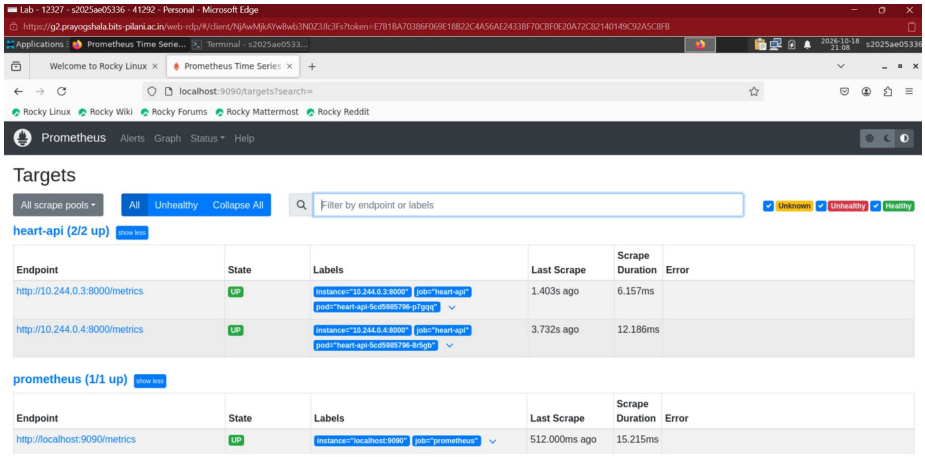


Figure 41. *Lab VM: Prometheus targets, both API pods UP.*

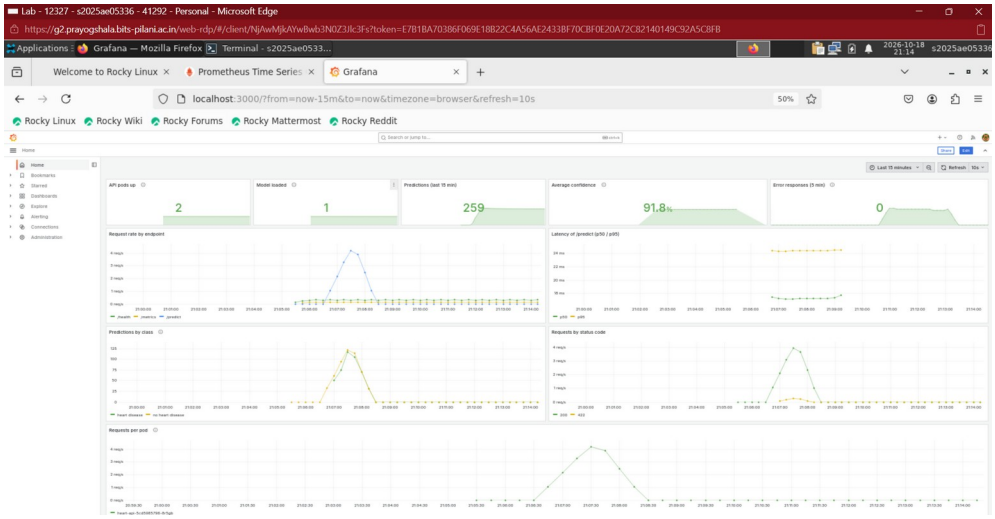


Figure 42. *Lab VM: Grafana dashboard after the traffic run (browser zoom reduced so the full dashboard fits).*

13. Challenges, limitations and lessons

- **Environment issues.** Python was not on PATH until VS Code was restarted; PowerShell blocked virtual-environment activation (fixed with a process-scoped execution policy); VS Code Restricted

Mode hid the kernel; Docker Desktop needed WSL. A stale requirements file caused a missing-MLflow error, solved by regenerating it.

- **Dependency pinning.** The serving image is pinned to scikit-learn 1.6.1, the version that produced the model, to avoid unpickling problems.
- **Kubernetes security check.** A named non-root user would not satisfy runAsNonRoot, so the image uses numeric UID 10001.
- **Scrape visibility.** With default log tails, health-probe lines hid the /predict lines; filtering the logs fixed the evidence.
- **Lab VM limits.** Lab sessions ended after about twenty minutes and the VM was replaced, which deleted the Docker images and the Minikube cluster; only the workspace folder survived. I rebuilt the cluster each time with the same README commands, wrapped in a small helper script kept in the workspace, and recorded the video in short segments.
- **Limitations.** Only 303 patients and a 61-row test set, so metric differences between models are within noise; the model is an educational exercise and not a clinical tool. The API has no authentication, Prometheus and Grafana store data in emptyDir volumes (lost when their pods are replaced) and the Minikube deployment is local. CI builds and smoke-tests the image, but deployment to the cluster is a manual `kubectl apply` because GitHub-hosted runners cannot reach a local Minikube.
- **Possible next steps.** Push the image to a registry and add a deploy job against a cloud cluster, add data-drift monitoring and alert rules, and calibrate and externally validate the model.

14. How to reproduce and where to find everything

Repository: https://github.com/vankudrv/heart-disease-mlops. The README lists the exact commands for each stage. The pipeline video is available in two copies, listed on the title page: the full-quality file (about 115 MB) on Google Drive, https://drive.google.com/file/d/1l1NyGt8uGgFhaOEbg-KFtx0vx-AS1yjG/view?usp=drive_link, and a compressed copy (about 20 MB) in the repository, https://github.com/vankudrv/heart-disease-mlops/raw/main/video/MLOPS_Assignment1_2025AE05336.mp4 (opens or downloads the file; GitHub does not preview large videos on its file page). It shows the architecture, the data and model choice, and the CI runs, then, recorded on the BITS lab VM, the tests, MLflow, the Docker container, the Kubernetes deployment and scaling, and the Prometheus and Grafana monitoring with live traffic.

Stage	Command
Train and track	python -m src.train ; mlflow ui
Tests and lint	python -m pytest ; python -m flake8 src tests
Run API locally	python -m uvicorn src.api.main:app --port 8000
Docker	docker build -t heart-disease-api:1.0 . ; docker run --rm -p 8000:8000 heart-disease-api:1.0
Kubernetes	minikube start --driver=docker ; minikube image load heart-disease-api:1.0 ; kubectl apply -k k8s/ ; minikube tunnel
Monitoring	kubectl apply -k monitoring/ ; kubectl port-forward -n monitoring svc/grafana 3000:3000

Table 6. *Reproduction commands.*

References

- Detrano, R. et al. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. American Journal of Cardiology, 64, 304-310.
- Dua, D. and Graff, C. UCI Machine Learning Repository: Heart Disease Data Set, https://archive.ics.uci.edu/dataset/45/heart+disease
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830.
- MLflow, FastAPI, Prometheus, Grafana, Kubernetes and Minikube official documentation.